

Proiect Procesarea Semnalelor

Petre Robert Cristian, Pascaru Dan-Alexandru

January 2026

1 Abstract

This paper presents the design of a custom communications protocol based on UDP. The core challenge addressed is the analysis and optimization for three distinct scenarios involving non-stationary data streams: video game server-client communication, satellite communication, and instant messaging.

These scenarios were selected as they represent the critical trade-offs required between latency, reliability, and spectral efficiency in modern communication systems.

The proposed solution is an adaptive, modular protocol featuring a six-layer pipeline, with certain layers being optional based on the specific use case: data source (raw data), compression (LZSS and Arithmetic Coding), encryption, error correction, transport and anomaly detection.

Protocol validation is conducted using real-world datasets from the European Space Agency (Mars Express mission), video game telemetry, online chat logs, and movie scripts. The results demonstrate the system's capacity for dynamic reconfiguration to optimize transmission according to specific channel constraints.

2 Rezumat

Această lucrare prezintă proiectarea unui protocol de comunicații bazat pe UDP. Problema centrală este analiza și optimizarea pentru trei scenarii, care reprezintă fluxuri de date non-staționare: comunicarea server-client în jocuri video, comunicare sateliți și mesagerie instantanee.

Aceste scenarii au fost alese deoarece reprezintă compromisul necesar între latență, fiabilitate și rata de compresie în sistemele de comunicații moderne.

Soluția propusă este un protocol adaptiv, modular, alcătuit dintr-un pipeline format din șase straturi, unele opționale: sursă (datele brute), compresie (LZSS și codare aritmetică), criptare, corecția erorilor, transport și detectare anomalii.

Validarea protocolului este realizată prin utilizarea seturilor de date reale provenite de la Agenția Spațială Europeană (misiunea Mars Express), date de telemetrie din jocuri video, din chat-uri online și scenarii de film, demonstrând

capacitatea sistemului de a se reconfigura dinamic pentru a optimiza transmisia în funcție de constrângerile canalului.

Contents

1	Abstract	1
2	Rezumat	1
3	Introducere	2
4	Compresie	4
4.1	LZSS (Lempel-Ziv-Storer-Szymanski)	5
4.2	Codare Aritmetică	5
4.3	Dialoguri Film	6
4.4	Chat Twitch	7
4.5	Telemetrie ESA	8
4.6	Comunicare Client-Server Minecraft	10
5	Criptare Date	12
6	Metode wavelet	13
6.1	Transformata Wavelet Discretă	13
6.2	Haar Wavelet	14
6.3	Daubechies wavelets	15
6.4	Symmlet wavelets	16
7	Analiza seriilor de timp	16
7.1	Extensia seriei de timp	17
7.2	Ferestre glisante și formarea matricilor de observații	17
7.3	MLE	18
7.4	Identificarea anomaliilor	18
7.5	Voting multi-wavelet	19
7.6	Evaluare și metrici de performanță	19
8	Concluzii	21

3 Introducere

Pentru a crea un protocol, în primul rând trebuie să stabilim ce fel de informație dorim să transmitem, iar asta va determina cum o transmitem.

Dacă datele trebuie să ajungă complet și în ordinea corectă, avem nevoie de un mecanism de acknowledgement sau de detectare și corectare erori. De exemplu, într-o mesagerie instantă, când trimiți un mesaj, te aștepti ca acel mesaj să fie garantat să ajungă la destinatar. Evident, poate ajunge cu o mică întârziere, însă ai garanția că o să ajungă într-un timp rezonabil. Acesta este un

caz în care prioritizezi fiabilitatea printr-un sistem de acknowledgement, chiar dacă viteza este sacrificată în proces.

Un alt scenariu în care ai nevoie de fiabilitate este comunicarea cu un satelit sau cu orice alt obiect aflat la o mare distanță. Aceasta este asigurată, în general, nu printr-un proces de acknowledgement, din cauza latenței ridicate, ci prin coduri de corectare a erorilor.

Putem avea și cazul opus, în care accentul este pus pe viteză, nu pe fiabilitate. Un exemplu ar fi jocurile video, unde într-un mediu dinamic un pachet pierdut și retransmis devine irelevant [17]. În aceste sisteme, starea lumii de joc trebuie actualizată în timp real, adesea prin delta compression pentru a trimite doar diferențele față de stările anterioare [18].

În toate aceste cazuri, trebuie să ținem cont și de dimensiunea datelor transmise și cum putem reduce această dimensiune. Compresia datelor este esențială din două motive:

- pentru optimizarea lățimii de bandă, în principal pentru a reduce costuri, însă este esențială în anumite cazuri, precum comunicarea cu obiecte îndepărtate sau cu mai mulți clienți.
- reducerea latenței, deoarece poate rezulta într-un timp de transmisie mai scurt [10, 11].

De asemenea, un aspect important este reprezentat de implementarea unor mecanisme de detectare a anomaliilor. În cadrul sistemelor de mesagerie, acestea sunt esențiale pentru filtrarea mesajelor de tip spam sau identificarea atacurilor cibernetice. Pentru comunicațiile prin satelit, monitorizarea permite detectarea degradării senzorilor sau a interferențelor cauzate de zgomotul cosmic, în timp ce în jocuri video, analiza fluxului de date asigură integritatea prin sisteme anti-cheat și permite declanșarea unui proces de resincronizare forțată în cazul în care decalajul dintre client și server este prea mare.

Prezenta lucrare analizează eficiența unor diverși algoritmi prin prisma acestor scenarii, evaluând cum pot îmbunătăți performanța globală a sistemului.

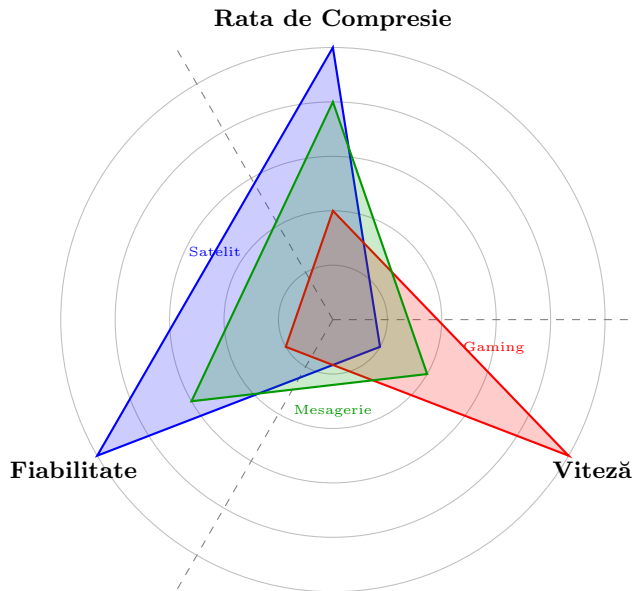


Figure 1: Aproximare a compromisului între cei trei factori în funcție de scenariu

4 Compresie

Scopul compresiei este reducerea dimensiunii datelor prin eliminarea redundanței. Am efectuat mai multe teste de performanță pe patru seturi de date reprezentative pentru scenariile din introducere, obiectivul fiind evaluarea eficienței algoritmilor implementați (LZSS și codarea aritmetică), în raport cu o soluție clasică (Huffman) și Burrows-Wheeler.

Accentul cade pe LZSS și codarea aritmetică, însă am inclus și Burrows-Wheeler [14] deoarece este o soluție modernă utilizată în metode de compresie populare, precum BZIP2 (se bazează pe transformarea Burrows-Wheeler combinată cu Move-To-Front și codare Huffman).

Alte metode de compresie care utilizează algoritmii menționați sunt:

- **DEFLATE (ZIP, PNG, GZIP):** utilizează o combinație de LZ77 [11] (precursorul direct al LZSS) și codare Huffman [10].
- **LZMA (7-Zip, .xz):** utilizează o variantă avansată de LZSS împreună cu *Range Coding* (o metodă funcțional identică cu codarea aritmetică).
- **LZ4:** reprezintă standardul actual pentru viteză, fiind utilizat în sisteme de fișiere și baze de date, bazându-se pe o implementare ultra-optimizată de LZ77.

4.1 LZSS (Lempel-Ziv-Storer-Szymanski)

LZSS este un algoritm de tip dicționar glisant, derivat din LZ77[11]. LZSS, formalizat de Storer și Szymanski în 1982[9], elimină redundanța structurală prin înlocuirea secvențelor de caractere care se repetă cu o referință către o apariție anterioară a aceleiași secvențe.

Diferența fundamentală față de LZ77 constă în faptul că LZSS utilizează o referință (perechea distanță-lungime) doar dacă lungimea șirului identificat este mai mare decât spațiul necesar pentru reprezentarea referinței propriu-zise. În caz contrar, simbolul este transmis ca un literal (caracter necomprimat). Această decizie este semnalată prin utilizarea unui flag de un bit, care precedă fiecare unitate de date. Un bit cu valoare 0 reprezintă un literal, un bit cu valoarea 1 o referință.

Eficiența algoritmului este determinată de doi parametri principali: dimensiunea ferestrei glisante, care reprezintă distanța maximă la care se poate face o referință și lungimea maximă a potrivirii. În cadrul testelor am menținut o lungime maximă de 255, iar pentru dimensiunea ferestrei am folosit valori de 8192 și 65536.

De asemenea am implementat două versiuni diferite: o versiune greedy, care imediat ce găsește o potrivire la poziția curentă creează o referință, și o versiune lazy, care amână crearea referinței, verificând dacă este posibilă o secvență mai lungă la poziția următoare.

Complexitate: $O(W \cdot N)$

Poate fi optimizat cu un hash table ceea ce ar rezulta într-o complexitate de $O(N)$.

4.2 Codare Aritmetică

Codarea aritmetică reprezintă o tehnică de compresie[8] ce reușește să se apropie de limita teoretică a entropiei lui Shannon mai mult decât codarea Huffman. În timp ce Huffman este constrâns să utilizeze un număr întreg de biți pentru fiecare simbol, codarea aritmetică poate atribui fracțiuni de biți pe simbol.

Algoritmul [12] mapează un flux de date într-un interval numeric din domeniul $[0, 1)$. Procesul începe cu un interval inițial $[0, 1)$ care este subdivizat succesiv în sub-intervale, proporțional cu probabilitatea de apariție a fiecărui simbol din alfabet.

Fiecare pas al procesului are următoarele etape:

1. **Calculul intervalului:** se determină lungimea intervalului curent
2. **Subdivizarea:** se restrânge intervalul pe baza probabilității de apariție a simbolului encodat
3. **Normalizarea:** pe măsură ce intervalul devine mai mic, se efectuează operații de re-scalare pentru a menține precizia numerică și a transmite biții determinați către fluxul de ieșire.

În cadrul acestei lucrări am ales să implementăm un model adaptiv. Acesta pornește cu o distribuție uniformă și actualizează probabilitățile pe măsură ce datele sunt procesate. Această abordare oferă două avantaje:

- Nu mai este necesară transmiterea tabelului de frecvențe.
- Modelul se poate adapta la schimbări de context.

Complexitate: $O(N)$

4.3 Dialoguri Film

Primul set de date utilizat în cadrul experimentelor este extras din *Cornell Movie-Dialogs Corpus*, un set de date ce cuprinde dialoguri din scenarii de film, disponibil pe platforma Kaggle [20]. Am ales acest set de date deoarece reprezintă un model al conversațiilor umane, fiind util pentru validarea protocolului în scenariul de mesagerie instantanee.

Spre deosebire de datele de telemetrie sau cele provenite din jocuri video, dialogurile de film prezintă o redundanță structurală mai scăzută, deoarece limbajul natural tinde să fie mai variat. Totuși, prezența unor structuri gramaticale repetitive și a unui vocabular finit permite algoritmului LZSS să identifice potriviri la nivel de cuvinte și expresii.

Utilizarea acestui set de date ne permite să evaluăm eficiența codării aritmetice adaptive în contextul textului lingvistic, unde distribuția probabilităților literelor se modifică pe măsură ce contextul conversației se schimbă. Rezultatele obținute pe acest set (vezi Tabelul 1) servesc ca punct de referință pentru performanța sistemului în condiții de utilizare zilnică a unei aplicații de chat.

Table 1: Benchmark

Algoritm	Dim. (bytes)	Ratio (%)	Timp (ms)	Viteză (MB/s)
Huffman	317,488	63.46%	48.44	9.85
Arithmetic	316,062	63.18%	419.46	1.14
LZSS (W=8192, L=255, Lazy=F)	269,116	53.79%	4,832.89	0.10
+ Huffman	256,694	51.31%	4,903.12	0.10
+ Arithmetic	254,892	50.95%	5,084.34	0.09
LZSS (W=65536, L=255, Lazy=F)	210,688	42.11%	24,263.48	0.02
+ Huffman	207,026	41.38%	24,186.79	0.02
+ Arithmetic	204,942	40.96%	24,340.01	0.02
LZSS (W=8192, L=255, Lazy=T)	258,053	51.58%	8,656.35	0.06
+ Huffman	247,597	49.49%	8,652.22	0.06
+ Arithmetic	245,638	49.10%	8,875.77	0.05
LZSS (W=65536, L=255, Lazy=T)	201,320	40.24%	50,432.36	0.01
+ Huffman	198,768	39.73%	49,656.20	0.01
+ Arithmetic	196,767	39.33%	50,466.41	0.01
BWT+MTF + Huffman	209,365	41.85%	326.45	1.46
BWT+MTF + Arithmetic	208,594	41.69%	672.36	0.71

Ceea ce iese în evidență în acest benchmark, dar și în următoarele, este eficiența de compresie a algoritmului LZSS. Putem ajunge la o rată de comprimare de aproximativ 40% utilizând *doar* LZSS, performanță care poate fi îmbunătățită puțin fără a sacrifica prea mult viteza cu ajutorul codării aritmetice.

Se poate observa o îmbunătățire semnificativă a ratei de comprimare când creștem W, care reprezintă dimensiunea ferestrei glisante. Evident, sacrificând viteza, algoritmul fiind în acest caz de zece ori mai lent.

Și mai remarcabil este faptul că BWT + MTF + Huffman are o rată de comprimare aproape la fel de bună, însă este de aproximativ 80 de ori mai rapidă.

4.4 Chat Twitch

Setul de date utilizat pentru acest scenariu provine din arhiva *Twitch Chat Dataset* de pe platforma Hugging Face [21].

Am ales acest set deoarece reprezintă un mediu de comunicare non-staționar și aparent haotic. Din punct de vedere lingvistic, redundanța este scăzută din cauza utilizării simbolurilor atipice și a link-urilor, însă din punct de vedere structural, setul prezintă ferestre de redundanță extrem de ridicată. Acest fenomen se datorează comportamentului specific utilizatorilor de pe această platformă, care repetă frecvent aceleași secvențe de caractere (emoticon-uri sau mesaje de cypypasta”). Analiza acestui set ne permite să observăm cum

reacționează algoritmi de dicționar în momentul în care fluxul de date alternează rapid între mesaje unice și blocuri masive de date identice.

Table 2: Benchmark

Algoritm	Dim. (bytes)	Ratio (%)	Timp (ms)	Viteză (MB/s)
Huffman	342,074	66.80%	45.04	10.84
Arithmetic	334,582	65.34%	429.90	1.14
LZSS (W=8192, L=255, Lazy=F)	273,061	53.32%	6,576.81	0.07
+ Huffman	261,096	50.99%	5,850.89	0.08
+ Arithmetic	258,970	50.57%	6,538.75	0.07
LZSS (W=65536, L=255, Lazy=F)	212,709	41.54%	29,724.68	0.02
+ Huffman	208,230	40.66%	29,957.25	0.02
+ Arithmetic	206,214	40.27%	30,134.82	0.02
LZSS (W=8192, L=255, Lazy=T)	266,221	51.99%	9,924.62	0.05
+ Huffman	255,042	49.80%	10,420.70	0.05
+ Arithmetic	252,981	49.40%	10,259.05	0.05
LZSS (W=65536, L=255, Lazy=T)	204,398	39.91%	49,156.46	0.01
+ Huffman	200,740	39.20%	53,328.21	0.01
+ Arithmetic	198,745	38.81%	49,616.02	0.01
BWT+MTF + Huffman	240,601	46.98%	342.13	1.43
BWT+MTF + Arithmetic	237,448	46.37%	714.25	0.68

LZSS + Arithmetic au o rata de comprimare asemanatoare cu cea observata pe setul de date anterior, insa performanta BWT + MTF + Huffman/Arithmetic este in mod clar inferioara.

4.5 Telemetrie ESA

Acest set de date provine din arhiva istorică a misiunii *Mars Express* a Agenției Spațiale Europene (ESA) și este disponibil public prin intermediul platformei Kaggle [19]. Datele cuprind înregistrări de telemetrie raportate de diverși senzori de la bordul navei spațiale, monitorizând parametri critici pe parcursul mai multor ani.

Am ales acest set de date pentru a valida protocolul în scenariul de comunicații spațiale, unde eficiența spectrală este vitală din cauza lățimii de bandă extrem de limitate. Particularitatea acestor date constă în structura lor ciclică și predictibilă: senzorii eșantionează aceiași parametri la intervale regulate, generând fluxuri binare cu un grad extrem de ridicat de redundanță structurală.

Table 3: Benchmark

Algoritm	Dim. (bytes)	Ratio (%)	Timp (ms)	Viteză (MB/s)
Huffman	297,978	28.42%	34.48	29.00
Arithmetic	269,178	25.67%	785.19	1.27
LZSS (W=8192, L=255, Lazy=F)	14,949	1.43%	125.26	7.98
+ Huffman	10,500	1.00%	126.48	7.91
+ Arithmetic	9,356	0.89%	137.66	7.26
LZSS (W=65536, L=255, Lazy=F)	14,889	1.42%	919.04	1.09
+ Huffman	8,258	0.79%	922.02	1.08
+ Arithmetic	7,086	0.68%	933.84	1.07
LZSS (W=8192, L=255, Lazy=T)	14,949	1.43%	175.06	5.71
+ Huffman	10,500	1.00%	176.44	5.67
+ Arithmetic	9,356	0.89%	187.31	5.34
LZSS (W=65536, L=255, Lazy=T)	14,887	1.42%	1,293.54	0.77
+ Huffman	8,256	0.79%	1,284.48	0.78
+ Arithmetic	7,084	0.68%	1,325.66	0.75
BWT+MTF + Huffman	134,809	12.86%	207,454.86	0.00
BWT+MTF + Arithmetic	9,460	0.90%	208,849.23	0.00

Aici avem un caz în care viteza de comprimare nu contează, ci mai degrabă rata de comprimare.

Un mod evident prin care am putea îmbunătăți eficiența comprimării ar fi crescând valoarea lui W . În scenariile prezentate până acum se poate observa o diferență enormă între rata de comprimare pentru $W=8192$ și $W=65536$. O întrebare naturală în această situație este următoarea: până la ce valoare a lui W putem observa îmbunătățiri în rata de comprimare?

Există următorul trade-off: pentru o dimensiune a ferestrei mai mare avem acces la mai multe tipare, însă referința noastră va ocupa mai mult spațiu (necesită mai mulți biți pentru a reprezenta distanța).

Pentru a evalua oportunitatea extinderii ferestrei de căutare de la $W = 2^n$ la $W = 2^{n+1}$, definim profitul informațional total P ca fiind diferența dintre biții economisiți prin substituție și costul referințelor introduse.

Fie:

- $M(W)$ numărul de potriviri (*matches*) identificate în fereastra W .
- L_{avg} lungimea medie a unei potriviri (în octeți).
- n numărul de biți necesari pentru reprezentarea *offset*-ului ($\lceil \log_2 W \rceil$).

Profitul pentru o fereastră de dimensiune 2^n este dat de:

$$P_n = M(2^n) \cdot (8L_{avg} - n) \quad (1)$$

Trecerea la o fereastră de dimensiune 2^{n+1} implică un cost suplimentar de 1 bit pentru fiecare referință, dar permite identificarea unui număr adițional de potriviri, $\Delta M = M(2^{n+1}) - M(2^n)$. Noul profit P_{n+1} devine:

$$P_{n+1} = (M(2^n) + \Delta M) \cdot (8L_{avg} - (n + 1)) \quad (2)$$

Condiția de rentabilitate $P_{n+1} > P_n$ se deduce după cum urmează:

$$\begin{aligned} (M(2^n) + \Delta M) \cdot (8L_{avg} - n - 1) &> M(2^n) \cdot (8L_{avg} - n) \\ M(2^n)(8L_{avg} - n) - M(2^n) + \Delta M(8L_{avg} - n - 1) &> M(2^n)(8L_{avg} - n) \\ \Delta M \cdot (8L_{avg} - n - 1) &> M(2^n) \end{aligned} \quad (3)$$

Astfel, putem extinde W cât timp:

$$\frac{\Delta M}{M(2^n)} > \frac{1}{8L_{avg} - n - 1} \quad (4)$$

De asemenea, după un anumit punct fereastra nu va mai avea loc în cache, punct în care degradarea vitezei va fi și mai vertiginoasă.

Trebuie menționat că pe acest set de date se observă cel mai clar limitarea codării Huffman, fiind observată o diferență semnificativă când folosim BWT împreună cu Huffman sau codarea aritmetică. Diferența în rata de comprimare este cel mai probabil datorată faptului că Huffman nu poate alocă mai puțin de un bit pentru un simbol.

4.6 Comunicare Client-Server Minecraft

Setul de date `minecraft_dataset.bin` a fost generat experimental prin capturarea traficului de rețea în timpul unei sesiuni active de joc. Metodologia de colectare a implicat interceptarea comunicării bidirecționale dintre client și server, asigurându-ne că mecanismul nativ de compresie al protocolului Minecraft a fost dezactivat.

Table 4: Benchmark

Algoritm	Dim. (bytes)	Ratio (%)	Timp (ms)	Viteză (MB/s)
Huffman	509,491	48.59%	55.36	18.06
Arithmetic	442,923	42.24%	834.55	1.20
LZSS (W=8192, L=255, Lazy=F)	266,919	25.46%	8,050.35	0.12
+ Huffman	234,507	22.36%	7,967.98	0.13
+ Arithmetic	232,129	22.14%	8,162.34	0.12
LZSS (W=65536, L=255, Lazy=F)	249,933	23.84%	46,941.70	0.02
+ Huffman	231,310	22.06%	46,538.21	0.02
+ Arithmetic	229,099	21.85%	47,225.04	0.02
LZSS (W=8192, L=255, Lazy=T)	254,111	24.23%	15,856.18	0.06
+ Huffman	223,244	21.29%	15,898.86	0.06
+ Arithmetic	220,911	21.07%	15,853.36	0.06
LZSS (W=65536, L=255, Lazy=T)	236,309	22.54%	94,649.33	0.01
+ Huffman	217,831	20.77%	95,711.87	0.01
+ Arithmetic	215,652	20.57%	95,410.01	0.01
BWT+MTF + Huffman	251,416	23.98%	18,649.66	0.05
BWT+MTF + Arithmetic	206,529	19.70%	19,514.89	0.05

Minecraft folosește DEFLATE (LZ77 + Huffman) pentru comprimare[11, 10]. Cel mai apropiat rezultat din tabel fiind LZSS + Huffman.

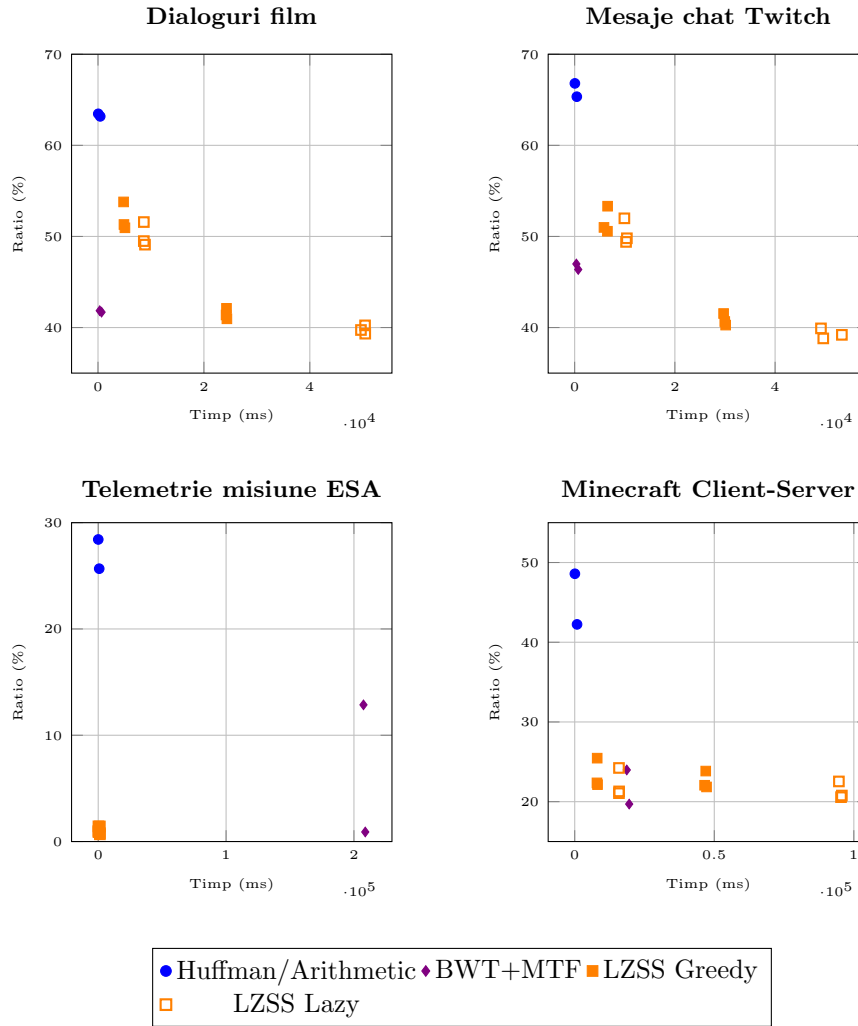


Figure 2: Eficiență vs. Timp (64 de configurații testate).

5 Criptare Date

Nu vom intra în detalii deoarece criptarea datelor reprezintă un subiect vast și nu constituie obiectul central al prezentei lucrări. Însă trebuie menționat că datele trebuie comprimate înainte de criptare, deoarece acest proces crește entropia datelor și limitează drastic capacitatea de comprimare.

Această ordine a operațiunilor poate introduce vulnerabilități subtile. Atacuri precum CRIME sau BREACH demonstrează că un atacator poate deduce conținutul datelor criptate observând variații în dimensiunea output-ului comprimat.

6 Metode wavelet

Un wavelet este o funcție oscilatorie de durată scurtă, utilizată în reprezentarea și analiza semnalelor. Aceste funcții joacă rolul de kernel de integrare în cadrul unei transformări integrale numite Transformata Wavelet.

Spre deosebire de Transformata Fourier, care oferă o localizare exactă în domeniul frecvenței, dar pierde informația temporală, Transformata Wavelet permite obținerea informației atât în domeniul timpului, cât și al frecvenței. Astfel, analiza wavelet realizează un compromis între rezoluția temporală și cea spectrală.

Rezoluția în timp și frecvență depinde de scala waveletului utilizat: cele mici oferă o bună localizare temporală, iar cele mari au o rezoluție mai bună în frecvență.

6.1 Transformata Wavelet Discretă

Transformata Wavelet Discretă (DWT) este o tehnică de analiză a semnalului la mai multe niveluri de rezoluție, bazată pe aplicarea succesivă a unor filtre wavelet. Prin aplicarea transformatei, informația este separată în componente care descriu comportamentul global al semnalului și variațiile locale, la diferite straturi de rezoluție.

La nivel matematic, DWT poate fi interpretată ca o convoluție între semnalul de intrare $x[n]$ și un filtru wavelet $g[n]$:

$$y[n] = (x * g)[n] = \sum_{k=-\infty}^{\infty} x[k] g[n - k]$$

Funcțiile wavelet sunt obținute prin scalarea și translația unei funcții de bază $\psi(t)$. În cazul unei scalări dyadice ($\lambda = 2$), forma generală a unei funcții wavelet este:

$$\psi_{j,k}(t) = \frac{1}{\sqrt{2^j}} \psi\left(\frac{t - k 2^j}{2^j}\right),$$

unde j reprezintă parametrul de scară, iar k poziția în timp.

Coeficienții obținuți în urma filtrării se împart în două categorii: coeficienți de aproximare și coeficienți de detaliu, în funcție de tipul de informație pe care o descriu. Coeficienții de aproximare surprind structura globală a semnalului și sunt utilizați pentru filtrarea la nivelurile următoare, în timp ce coeficienții de detaliu evidențiază variațiile locale și energia asociată tranzițiilor rapide.

Această separare este posibilă datorită faptului că funcțiile wavelet pot forma baze ortonormate complete, ceea ce permite descompunerea semnalului într-o componentă de aproximare și una de detaliu, fără pierdere de informație.

În practică, această descompunere este realizată prin utilizarea a două filtre

discrete:

$$c[k] = \sum_n x[n] h[n - 2k], \quad \text{unde } h[n] \text{ este filtrul de aproximare}$$

$$d[k] = \sum_n x[n] g[n - 2k], \quad \text{unde } g[n] \text{ este filtrul de detaliu}$$

Energia semnalului este garantat conservată datorită proprietăților de ortonormalitate, relația fiind:

$$\sum_{n=-\infty}^{\infty} |x[n]|^2 = \sum_k |c_{i_0, k}|^2 + \sum_{i=i_0}^{\infty} \sum_k |d_{i, k}|^2$$

6.2 Haar Wavelet

Haar wavelet este cel mai simplu wavelet ortogonal, filtrele sale realizează o mediere și o diferențiere locală a semnalului. Are capacitatea de a detecta variații bruște și tranziții rapide, fiind eficient din punct de vedere computațional. Totodată, nu este potrivit pentru aproximarea semnalelor netede, având un singur moment nul.

Funcția de scalare asociată waveletului Haar este definită prin:

$$\phi(t) = \begin{cases} 1, & 0 \leq t < 1, \\ 0, & \text{altfel.} \end{cases}$$

Pentru orice pereche de întregi $n, k \in \mathbb{Z}$, funcțiile Haar sunt obținute prin scalarea și translația waveletului mamă:

$$\psi_{n, k}(t) = 2^{n/2} \psi(2^n t - k), \quad t \in \mathbb{R},$$

unde waveletul mamă este definit ca:

$$\psi(t) = \begin{cases} 1, & 0 \leq t < \frac{1}{2}, \\ -1, & \frac{1}{2} \leq t < 1, \\ 0, & \text{altfel.} \end{cases}$$

Funcțiile de scalare satisfac următoarele relații:

$$\phi(t) = \phi(2t) + \phi(2t - 1),$$

$$\psi(t) = \phi(2t) - \phi(2t - 1).$$

În implementarea discretă a Transformatei Wavelet Discrete, aceste relații conduc la definiția a două filtre finite, utilizate pentru calculul coeficienților:

$$h = \frac{1}{\sqrt{2}}[1, 1], \quad g = \frac{1}{\sqrt{2}}[1, -1].$$

6.3 Daubechies wavelets

Waveleturile Daubechies reprezintă o familie de waveleturi ortogonale cu suport compact, caracterizate printr-un număr p de momente nule.

Spre deosebire de waveletul Haar, waveleturile Daubechies nu admit o expresie simplă pentru funcția $\psi(t)$. În schimb, acestea sunt definite prin coeficienții filtrului de scalare $h[n]$, care satisfac condiții de ortonormalitate și reconstrucție perfectă.

Transformata Fourier a filtrului de scalare este un polinom trigonometric de forma:

$$\hat{h}(\omega) = \sum_{n=0}^{N-1} h[n] e^{-in\omega}.$$

Pentru a asigura existența a p momente nule ale waveletului, filtrul trebuie să conțină un factor de forma:

$$\left(\frac{1 + e^{-i\omega}}{2} \right)^p,$$

care introduce zerouri de ordin p la $\omega = \pi$. Astfel, filtrul de scalare devine:

$$\hat{h}(\omega) = \sqrt{2} \left(\frac{1 + e^{-i\omega}}{2} \right)^p R(e^{-i\omega}),$$

unde polinomul $R(e^{-i\omega})$ este ales astfel încât să fie satisfăcute condițiile de ortonormalitate:

$$|\hat{h}(\omega)|^2 + |\hat{h}(\omega + \pi)|^2 = 2.$$

Această construcție conduce la filtre finite de lungime $N = 2p$, utilizate în DWT.

Filtrul de detaliu este determinat direct din filtrul de scalare prin relația de conjugare în oglindă. Așadar din coeficienții filtrului de aproximare $h[n]$, se pot afla cei de detaliu $g[n]$ prin:

$$g[n] = (-1)^n h[N - 1 - n],$$

unde N reprezintă lungimea filtrului de scalare.

Coeficienții filtrului Daubechies pentru $p = 2$

n	$h[n]$
0	0.482962913145
1	0.836516303738
2	0.224143868042
3	-0.129409522551

$p = 2$

Coeficienții filtrului Daubechies pentru $p = 2$ și $p = 8$

n	$h[n]$	n	$h[n]$
0	0.230377813309	0	0.054415842243
1	0.714846570553	1	0.312871590914
2	0.630880767930	2	0.675630736297
3	-0.027983769417	3	0.585354683654
4	-0.187034811719	4	-0.015829105256
5	0.030841381836	5	-0.284015542962
6	0.032883011667	6	0.000472484574
7	-0.010597401785	7	0.128747426620
$p = 4$		8	-0.017369301002
		9	-0.044088253930
		10	0.013981027917
		11	0.008746094047
		12	-0.004870352993
		13	-0.000391740373
		14	0.000675449406
		15	-0.000117476784
		$p = 8$	

6.4 Symmlet wavelets

Waveleturile Symmlet reprezintă o variantă îmbunătățită a waveleturilor Daubechies, concepută pentru a reduce asimetria acestora. Asimetria apare în cazul waveleturilor Daubechies deoarece sunt construite folosind o rădăcină de fază minimă a filtrului de scalare, ceea ce conduce la o concentrare a energiei spre un capăt al suportului.

Pentru a obține simetrie sau antisimetrie, filtrele Symmlet sunt construite prin optimizarea alegerii rădăcinii polinomului asociat filtrului de scalare, astfel încât faza să fie aproape liniară.

Similar wavelet-urile Symmlet sunt ortogonale cu suport compact și un număr p de momente nule. Lungimea filtrului de scalare rămâne minimă pentru numărul de momente nule impus, iar coeficienții sunt reali.

7 Analiza seriilor de timp

Pentru a detecta anomalii în fluxurile de date analizate, am ales să utilizăm un algoritm inspirat din DWT-MLEAD [1], prin care avem ca obiectiv să punem în evidență capacitatea waveleturilor de a localiza energia semnalului simultan în timp și frecvență. Metoda se bazează pe combinarea DWT cu estimarea prin verosimilitate maximă (MLE), aplicată asupra coeficienților de detaliu și aproximare, cu scopul de a modela statistic comportamentul normal al semnalului pe mai multe niveluri temporale și a identifica structurile anormale.

Coeficienții filtrului Symmlet pentru $p = 3$ și $p = 4$

n	$h[n]$	n	$h[n]$
0	0.035226291882	0	-0.075765714789
1	-0.085441273882	1	-0.029635527645
2	0.135011020010	2	0.497618667632
3	0.459877502119	3	0.803738751805
4	0.806891509313	4	0.297857795605
5	0.332670552950	5	-0.099219543576
$p = 3$		6	-0.012603967262
		7	0.032223100604
		$p = 4$	

7.1 Extensia seriei de timp

Transformata wavelet discretă impune ca lungimea semnalului analizat să fie o putere a lui 2. Astfel, o serie de timp de lungime n este extinsă până la $m = 2^{\lceil \log_2 n \rceil}$, prin copierea în oglindă a ultimului segment al semnalului. Această metodă de extensie păstrează continuitatea locală a semnalului și evită introducerea unor discontinuități sau informații artificiale care ar putea afecta coeficienții. Anomaliile identificate în zona extinsă nu sunt luate în considerare în evaluarea finală.

7.2 Ferestre glisante și formarea matricilor de observații

Pentru a surprinde dependențele temporale dintre coeficienții wavelet și evoluția lor, trebuie să îi grupăm într-o formă favorabilă aplicării unui model statistic. Plecăm de la ipoteza că, într-o porțiune a seriei de timp lipsită de anomalii, coeficienții tind să urmeze distribuții concentrate în jurul unui comportament mediu stabil, din punct de vedere probabilistic.

În acest scop folosim o tehnică standard de tip *ferestre glisante* (sliding windows). Pentru un nivel l al transformării wavelet, considerăm separat coeficienții de detaliu d_l și coeficienții de aproximare c_l , glisând o fereastră de dimensiune w_l peste secvența de coeficienți și construim câte o matrice de observații:

$$D^{(l)} \in \mathbb{R}^{(N_l - w_l + 1) \times w_l}, \quad C^{(l)} \in \mathbb{R}^{(N_l - w_l + 1) \times w_l},$$

unde fiecare linie reprezintă un vector de observație format din w_l coeficienți consecutivi, iar N_l este numărul de coeficienți disponibili la nivelul l , respectiv $l \in \{l_{start}, \dots, L\}$, cu $L = \log_2(m)$ fiind nivelul maxim al transformării wavelet.

Dimensiunea ferestrei este aleasă dependent de nivel, conform:

$$w_l = \max\{2, l - l_{start} + 1\},$$

unde l_{start} reprezintă nivelul de la care începem analiza. Această alegere permite ca, pe măsură ce coborâm spre scări mai mari, fereastra să capteze un context temporal suficient de mare, cu un cost computațional potrivit.

7.3 MLE

Cu ajutorul matricilor $D^{(l)}$ și $C^{(l)}$, putem modela statistic comportamentul coeficienților wavelet prin intermediul unei distribuții Gaussiene multivariate. Fie $X \in \mathbb{R}^{n \times w_l}$ o astfel de matrice de observații, unde fiecare linie

$$\mathbf{x}_i \in \mathbb{R}^{w_l}$$

reprezintă o fereastră, iar $n = N_l - w_l + 1$ este numărul total de ferestre obținute la nivelul l .

Presupunem că observațiile obținute prin ferestrele glisante sunt independente și aparțin unei variabile aleatoare multivariate distribuite normal:

$$\mathbf{x}_i \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}),$$

Parametrii distribuției, adică vectorul medie $\boldsymbol{\mu} \in \mathbb{R}^{w_l}$ și matricea $\boldsymbol{\Sigma} \in \mathbb{R}^{w_l \times w_l}$, sunt calculați ca media pe coloane și matricea de covarianță a matricii X . Pentru fiecare observație $\mathbf{x}_i$ se calculează log-verosimilitatea față de distribuția Gaussiană multivariată:

$$\log p(\mathbf{x}_i) = -\frac{1}{2} \log \det(2\pi\boldsymbol{\Sigma}) - \frac{1}{2}(\mathbf{x}_i - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x}_i - \boldsymbol{\mu}).$$

Acest procedeu este aplicat atât asupra matricilor $D^{(l)}$, cât și $C^{(l)}$, obținându-se astfel o valoare de log-verosimilitate pentru fiecare fereastră de coeficienți.

7.4 Identificarea anomaliilor

Pe baza valorilor de log-verosimilitate obținute pentru fiecare fereastră de coeficienți, identificăm anomalități prin compararea acestora cu o cuantilă empirică $\epsilon \in (0, 1)$. Mai precis, pentru fiecare nivel l și pentru fiecare matrice de observații $X \in \{D^{(l)}, C^{(l)}\}$, calculăm cuantila ϵ a distribuției, iar ferestrele pentru care

$$\log p(\mathbf{x}_i) < z_\epsilon$$

sunt marcate ca fiind potențial anormale. Valoarea parametrului ϵ este aleasă empiric, observându-se că anomaliile reale corespund unor valori extrem de mici.

Având în vedere că Transformata Wavelet Discretă este o transformare decimată, fiecare coeficient de la nivelul l corespunde unui interval de lungime 2^l din seria de timp originală. Prin urmare, o observație anormală identificată la nivelul l nu poate fi asociată unui singur eșantion temporal, ci unui întreg interval. Pentru a reflecta acest aspect, introducem un vector de evidență

$$\mathbf{h} \in \mathbb{R}^m,$$

unde m este lungimea seriei de timp extinse. Pentru fiecare observație marcată ca anormală la nivelul l , incrementăm cu o unitate toate pozițiile corespunzătoare intervalului

$$[i \cdot 2^l, \min((i+1) \cdot 2^l, m)).$$

Acest procedeu permite acumularea dovezilor de anomalie provenite de la diferite niveluri de rezoluție și din ambii tipuri de coeficienți. Pentru a elimina evenimentele izolate, posibil generate de zgomot, sunt păstrate doar pozițiile pentru care

$$h_j \geq 2,$$

ceea ce implică că o anomalie este susținută de cel puțin două detecții independente.

Pozițiile rămase sunt grupate în clustere temporale astfel încât distanța dintre două poziții consecutive din același cluster să nu depășească un prag d_{max} . Pentru fiecare cluster C_k se calculează un scor total

$$S_k = \sum_{j \in C_k} h_j.$$

Un cluster este considerat anomal dacă scorul său depășește pragul B , ales empiric. În acest caz, poziția finală a anomaliei este determinată ca media ponderată a indicilor din cluster:

$$\mu(C_k) = \frac{\sum_{j \in C_k} j \cdot h_j}{\sum_{j \in C_k} h_j}.$$

7.5 Voting multi-wavelet

În varianta originală a algoritmului DWT-MLEAD, detecția anomaliilor se realizează utilizând o singură familie de waveleturi. Am extins algoritmul prin introducerea unui sistem democratic de alegere a anomaliilor finale. Ideea de bază constă în aplicarea independentă a algoritmului de detecție pentru mai multe tipuri de waveleturi (Haar, Db4 și Sym4), fiecare producând o mulțime proprie de anomalii candidate. Datorită proprietăților diferite ale waveleturilor, detecțiile pot varia ușor de la un wavelet la altul. Două detecții sunt considerate echivalente dacă distanța dintre ele este mai mică decât o toleranță temporală T . O anomalie este acceptată în rezultatul final doar dacă este susținută de cel puțin un număr minim de waveleturi. În cazul nostru am utilizat 3 waveleturi, deci am considerat că 2 voturi sunt suficiente pentru a constata o anomalie.

7.6 Evaluare și metrici de performanță

Evaluarea a fost realizată utilizând două tipuri de date. Primul set constă într-un dataset construit din fișierele `minecraft-dataset.bin`, `twitch-chat_raw.txt`, `chat_data_for_csharp.txt`. Al doilea set de date utilizat pentru evaluare este dataset-ul standard NAB (Numenta Anomaly Benchmark) [5], folosit pentru comparații suplimentare și validare externă a metodei propuse.

Seriile din dataset-ul construit au fost împărțite în segmente de dimensiune fixă (1024 bytes), fiecare segment fiind procesat printr-un pipeline de compresie format din algoritmul LZSS urmat de codare aritmetică. Raportul de compresie obținut pentru fiecare segment a fost utilizat drept indicator al comportamentului local al semnalului.

Etichetarea anomaliilor a fost realizată folosind o metodă bazată pe cuantile, fără a presupune o distribuție gaussiană a datelor. Pragurile de anomalie au fost stabilite la nivelurile inferioare și superioare ale distribuției rapoartelor de compresie. Au fost analizate două configurații:

- prag de anomalie de 1% (0.5% percentila inferioară și 0.5% percentila superioară);
- prag de anomalie de 2% (1% percentila inferioară și 1% percentila superioară).

Modelul a fost antrenat și evaluat pe dataset-ul construit, iar selecția parametrilor a fost realizată prin optimizarea performanței globale. Parametrii analizați includ pragul de sensibilitate ϵ , dimensiunea ferestrei B , adâncimea maximă d_{max} și nivelul inițial l_{start} .

Evaluarea pe dataset-ul NAB a fost utilizată exclusiv pentru a compara varianta implementată (*DWT_Custom*) cu implementarea standard (*DWT*), având ca scop validarea corectitudinii și comportamentului propriei implementări în raport cu cea originală, într-un mediu realist ce include atât date reale, cât și date sintetice de trafic.

Performanța a fost evaluată folosind metricile Precision, Recall și F1-score:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN},$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Performanța algoritmului DWT_Custom pe datasetul cu compresii

Anomalii (%)	Toleranță	ϵ	B	d_{max}	l_{start}	F1 Score	Precision	Recall
1	10	0.0100	13	3	1	0.7017	0.8723	0.6858
1	5	0.0100	13	3	1	0.6983	0.8582	0.6839
2	10	0.0211	5	7	1	0.6078	0.7602	0.6957
2	5	0.0159	9	11	1	0.5820	0.7419	0.6807

Rezultate obținute pe NAB dataset

Algorithm	Tolerance	ϵ	B	d_{max}	l_{start}	F1 Score	Precision	Recall
DWT_Custom	50	0.0133	23	9	1	0.3917	0.3573	0.5440
DWT	50	0.01	20	6	3	0.3940	0.3327	0.6227
DWT_Custom	10	0.0149	9	1	1	0.2431	0.3113	0.3307
DWT	10	0.01	17	5	2	0.2339	0.1899	0.4155

Rezultatele confirmă că performanța algoritmului bazat pe DWT este puternic influențată de alegerea pragurilor percentile și de densitatea anomaliilor din semnal. Pentru configurații cu praguri percentile stricte, corespunzătoare unei proporții reduse de anomalii, 1%, algoritmul reușește să identifice eficient

deviațiile extreme ale comportamentului normal. Acest comportament este evident atât pe setul cu compresii, unde distribuția anomaliilor este controlată, cât și pe datasetul NAB, în special pentru valori mai mari ale toleranței.

Pe măsură ce proporția de anomalii crește, performanța scade semnificativ. Estimarea bazată pe ferestre glisante și distribuții Gaussiene presupune o separare clară între comportamentul normal și cel anormal. În prezența unor anomalii dense sau consecutive, probabilitatea apariției mai multor erori adiacente crește, ceea ce conduce fie la etichetări excesive, fie la coruperea scorurilor de verosimilitate, afectând negativ detecția.

8 Concluzii

În urma documentării, experimentelor și a procesului de proiectare, concluzia fundamentală a acestei lucrări este că un protocol de comunicare universal optim este imposibil de realizat.

În funcție de scenariul de utilizare sunt necesare mecanisme diferite de transmitere a informației și de comprimare a informației; nu putem avea un protocol care le utilizează pe toate concomitent, ci mai degrabă o serie de protocoale diferite cu anumite puncte în comun, cum ar fi structura pachetelor, poate unii algoritmi de compresie sau poate mecanismul de transmisie sau detecție a anomaliilor. Așadar, trebuie să facem un compromis în funcție de contextul tehnic.

Codarea aritmetică exemplifică cel mai bine acest lucru: oferă o densitate informațională superioară, sacrificând viteza, însă aplicabilitatea sa este limitată chiar și în domenii care ar beneficia de o densitate informațională mai mare, precum designul seturilor de instrucțiuni (ISA). Acest lucru se datorează naturii strict secvențiale a algoritmului, care previne execuția paralelă și pipelining-ul instrucțiunilor.

Ca direcții viitoare de dezvoltare, considerăm explorarea unor tehnici moderne care încearcă să rezolve acest compromis între viteză și eficiență, precum:

- **Asymmetric Numeral Systems (ANS) [13]:** O soluție care promite să combine viteza codării Huffman cu rata de compresie a codării aritmetice.
- **Finite State Entropy (utilizat în Zstd) [16]:** Pentru o decompresie ultra-rapidă prin tabele precalculate.
- **Brotli [15]:** Pentru optimizarea transmisiunilor de text scurt, prin utilizarea dicționarelor statice.

Rezultatele obținute în această lucrare evidențiază și limitele abordărilor statistice clasice utilizate în detecția anomaliilor. Algoritmii bazați pe ipoteze de staționaritate, distribuții parametrice simple și ferestre glisante fixe tind să își piardă calitatea de clasificare în prezența seriilor temporale complexe. În acest sens, direcțiile moderne de cercetare propun modele mult mai expresive, care sunt capabile să învețe reprezentări latente adaptative direct din date. De exemplu, lucrări recente bazate pe arhitecturi de tip Transformer pentru

dectecția anomaliilor în serii temporale [6] exploatează mecanisme de atenție pentru a modela dependențe pe termen lung fără a impune ferestre fixe, oferind o adaptabilitate superioară față de metodele statistice tradiționale. În direcția opusă, abordările probabilistice avansate și autoencoderele temporale noi [7] permit o separare mai fină între variațiile normale și comportamentele anormale prin învățarea unor spații latente bogate, capabile să surprindă atât structuri locale, cât și globale ale semnalului.

Comparativ cu metoda propusă în această lucrare, astfel de modele moderne pot oferi:

- o toleranță crescută la anomalii consecutive sau dense, prin modelarea explicită a contextului temporal extins;
- eliminarea necesității unor praguri percentile fixe, prin scoruri de anomalie învățate end-to-end;
- o capacitate superioară de generalizare în scenarii reale, caracterizate prin non-staționaritate și variabilitate ridicată.

References

- [1] M. Thill, W. Konen, and T. Bäck, “*Time Series Anomaly Detection with Discrete Wavelet Transforms and Maximum Likelihood Estimation.*”, Proceedings of the International Conference on Time Series and Forecasting (ITISE), 2017.
- [2] S. Mallat, “*A Wavelet Tour of Signal Processing: The Sparse Way*”, Academic Press, 3rd Edition, 2008.
- [3] E. Foufoula-Georgiou and P. Kumar (eds.), “*Wavelets in Geophysics*”, Wavelet Analysis and Its Applications, Vol. 4, Academic Press, 1994.
- [4] Wikipedia, “*Wavelet transform*”, “*Discrete wavelet transform*”, “*Daubechies wavelet*”, “*Haar wavelet*”, “*Multivariate normal distribution*”.
- [5] Numenta Anomaly Benchmark (NAB), “*Numenta Anomaly Benchmark (NAB)*”.
- [6] Transformer-based Anomaly Detection for Time Series, “*Transformer-based Anomaly Detection for Time Series*”, arXiv:2212.13558, 2022.
- [7] Probabilistic Deep Learning for Time Series Anomaly Detection, “*Probabilistic Deep Learning for Time Series Anomaly Detection*”, arXiv:2503.03729, 2025.
- [8] I. H. Witten, R. M. Neal, and J. G. Cleary, “Arithmetic coding for data compression,” *Communications of the ACM*, vol. 30, no. 6, pp. 520–540, Jun. 1987.

- [9] J. A. Storer and T. G. Szymanski, “Data Compression via Textual Substitution,” *Journal of the ACM*, vol. 29, no. 4, pp. 928–951, Oct. 1982.
- [10] D. A. Huffman, “A Method for the Construction of Minimum-Redundancy Codes,” *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, Sep. 1952.
- [11] J. Ziv and A. Lempel, “A Universal Algorithm for Sequential Data Compression,” *IEEE Transactions on Information Theory*, vol. 23, no. 3, pp. 337–343, May 1977.
- [12] M. Nelson, “Data Compression With Arithmetic Coding,” *Mark Nelson’s Blog*, Oct. 2014. [Online].
- [13] J. Duda, “Asymmetric Numeral Systems: Entropy coding combining speed of Huffman coding with compression rate of arithmetic coding,” *arXiv preprint arXiv:1311.2540*, 2013.
- [14] M. Burrows, and D. J. Wheeler, “A Block-sorting Lossless Data Compression Algorithm,” SRC Research Report 124, 1994.
- [15] J. Alakuijala and Z. Szabadka, “Brotli Compressed Data Format,” *IETF RFC 7932*, Jul. 2016.
- [16] Y. Collet, “Finite State Entropy: A new breed of entropy coder,” *Fast Compression Blog*, Dec. 2013. [Online].
- [17] G. Fiedler, “Game Networking,” *Gaffer on Games*, 2016. [Online].
- [18] J. Fedor, “Quake III Arena Networking,” *Jiri Fedor’s Website*, 2000. [Online].
- [19] S. Mahoney, *ESA Anomaly Dataset*, Kaggle, 2024, <https://www.kaggle.com/datasets/sammahoney/esa-anomaly-dataset>
- [20] Cornell University, *Movie-Dialogs Corpus*, 2011. <https://www.kaggle.com/datasets/Cornell-University/movie-dialog-corpus>
- [21] https://huggingface.co/datasets/lparkourer10/twitch_chat